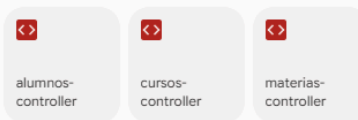


DIAGNÓSTICO:



identificá qué lógica se repite y propondría extraer a un helper, sin escribir código todavía. Listame los candidatos ordenados por cuánto código ahorran.

Al analizar los tres controladores ( `alumnos-controller.js` , `cursos-controller.js` y `materias-controller.js` ), queda claro que comparten una estructura casi idéntica. Prácticamente todo el manejo de las peticiones HTTP y el formateo de las respuestas se repite en cada entidad.

A continuación, te presento los candidatos a ser extraídos en funciones o clases *helper* (o *middleware*), ordenados de mayor a menor según el volumen de código duplicado que ahorrarían:

### 1. El bloque de manejo de errores en los bloques `catch` (Máximo ahorro)

Cada uno de los endpoints de todos los controladores tiene un bloque `catch` que hace exactamente lo mismo: hace un `console.log(error)` y responde con un código de estado ( `BAD_REQUEST` o `INTERNAL_SERVER_ERROR` ) junto con el mensaje del error.

- o **Lógica repetida:**

```
``js
catch (error) {
  console.log(error);
  res.status(StatusCode.INTERNAL_SERVER_ERROR).send( Error: ${error.message} );
}
```

### 2. Validación de coincidencia de IDs en solicitudes `PUT`

El método `PUT` de los tres controladores contiene exactamente la misma validación lógica para asegurarse de que el ID enviado en la URL coincida con el ID enviado dentro del cuerpo ( `body` ) de la petición.

- o **Lógica repetida:**

```
``js
if (entity.id && parseInt(entity.id) !== id) {
  return res.status(StatusCode.BAD_REQUEST).send( El id de la URL (${id}) no coincide con el id del body (${entity.id}). );
}
```
- o **Propuesta:** Extraer esto a un helper de validación, por ejemplo: `validateIdMatch(req, res)` . Esto limpia la parte superior de cada endpoint de actualización.

### 3. Orquestador de Respuestas Estándar (Control de éxito/error de base de datos)

Los bloques `try` de los métodos `GET` (por id), `POST` , `PUT` y `DELETE` siguen un patrón condicional idéntico: llaman al servicio, verifican si el resultado es válido (si no es nulo o si las filas afectadas son mayores a 0) y envían un código de estado u otro.

- o **Lógica repetida:**
  - o **GET por ID:** Si el objeto existe manda `OK (200)` con el JSON, si no, manda `NOT_FOUND (404)` .
  - o **PUT / DELETE:** Si las filas afectadas/eliminadas son distintas de 0 manda `OK (200)` , si no, manda `NOT_FOUND (404)` .
  - o **POST:** Si el `newId > 0` manda `CREATED (210)` , si no, `BAD_REQUEST (400)` .
- o **Propuesta:** Crear un helper de respuestas genéricas (ej: `sendEntityResponse(res, data, notFoundMessage)` o `sendMutationResponse(res, rowsAffected)` ). Evitaría tener que escribir los mismos `if / else` de HTTP en cada endpoint de cada controlador.

### Un paso más allá (Estrategia Avanzada)

Si notas que la estructura de los controladores `CursosController` y `MateriasController` es 100% idéntica en su comportamiento CRUD básico, el mayor candidato a ahorro a largo plazo no es solo un helper individual, sino un **Controlador Base Genérico**.

## Prompt:



log-helper

### # ROL

Actúa como desarrollador de node.js senior

### # CONTEXTO

Quiero extraer lógica repetida a funciones reutilizables (helpers). El primero que quiero desarrollar es un Helper de respuestas (responderOk, responderNotFound, responderError...) para unificar el formato de las respuestas HTTP que hoy está copiado en cada endpoint de los controllers. El segundo es Helper de fechas: sacar calcularEdad / agregarEdad, que hoy viven adentro de alumnos-service.js, a un helper reutilizable. Ese cálculo no es lógica de "alumnos": es una utilidad de fechas que mañana podrían usar otras entidades. Tu trabajo es darte cuenta de que no pertenece al service y mudarlo a src/helpers/.

### # TAREA

Necesito que reemplaces las repeticiones por llamados a los helpers.

### # RESTRICCIONES

el helper tiene que ser una función pura / un módulo independiente, exportado con ES modules, y no debe cambiar la respuesta HTTP que ya devuelven los endpoints

### # ITERACIÓN

Explicame donde poner el helper y cómo nombrarlo según la convención del proyecto (mirá cómo está hecho LogHelper: clase con métodos estáticos vs. funciones sueltas).  
Coherencia > "lo más moderno"

## Respuesta:

# 1. Estructura y Ubicación de los Helpers

Crearemos los archivos dentro de `src/helpers/`:

[src/helpers/response-helper.js](#)

## JavaScript

None

```
import { StatusCodes } from 'http-status-codes';

class ResponseHelper {
  ok = (res, data) => {
```

```

        return res.status(StatusCode.OK).json(data);
    }

    created = (res, data) => {
        return res.status(StatusCode.CREATED).json(data);
    }

    badRequest = (res, message) => {
        return res.status(StatusCode.BAD_REQUEST).send(message);
    }

    badRequestJson = (res, data) => {
        return res.status(StatusCode.BAD_REQUEST).json(data);
    }

    notFound = (res, message) => {
        return res.status(StatusCode.NOT_FOUND).send(message);
    }

    internalServerError = (res, message) => {
        return
        res.status(StatusCode.INTERNAL_SERVER_ERROR).send(message);
    }
}

// Exportamos la instancia única tal como en log-helper.js
export default new ResponseHelper();

```

src/helpers/date-helper.js

*(Aquí es donde deben vivir calcularEdad y agregarAnios, extraídos de alumnos-service.js)*

## JavaScript

None

```

class DateHelper {
    calcularEdad = (fechaNacimiento) => {
        const hoy = new Date();
        const cumpleanos = new Date(fechaNacimiento);
        let edad = hoy.getFullYear() - cumpleanos.getFullYear();
        const mes = hoy.getMonth() - cumpleanos.getMonth();
    }
}

```

```

        if (mes < 0 || (mes === 0 && hoy.getDate() <
cumpleaños.getDate())) {
            edad--;
        }
        return edad;
    }

    agregarAños = (fecha, años) => {
        const resultado = new Date(fecha);
        resultado.setFullYear(resultado.getFullYear() + años);
        return resultado;
    }
}

// Exportamos la instancia única
export default new DateHelper();

```

## 2. Refactorización de los Controladores

Consumiendo el singleton mediante `ResponseHelper.metodo(...)`, la sintaxis de invocación queda limpia y la arquitectura unificada.

`alumnos-controller.js`

### JavaScript

None

```

import { Router } from 'express';
import AlumnosService from '../services/alumnos-service.js'
import Alumno from '../entities/alumno.js'
import ResponseHelper from '../helpers/response-helper.js'

const router = Router();
const currentService = new AlumnosService();

router.get('/test-insert', async (req, res) => {
    console.log('/test-insert');
    try {
        const nuevoAlumno = new Alumno('Willy', 'Wonka', 1,
'2005-07-15', true);
        console.log('Objeto Alumno creado desde código:',
nuevoAlumno);
    }
}

```

```

        const newId = await currentService.createAsync(nuevoAlumno);
        if (newId > 0) {
            return ResponseHelper.created(res, {
                message : `Se creó el alumno desde código con id:
${newId}`,
                alumno  : nuevoAlumno,
                newId    : newId
            });
        } else {
            return ResponseHelper.badRequestJson(res, { message: 'No
se pudo crear el alumno.' });
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.get('', async (req, res) => {
    try {
        console.log(`AlumnosController.get`);
        const returnArray = await currentService.getAllAsync();
        if (returnArray != null){
            return ResponseHelper.ok(res, returnArray);
        } else {
            return ResponseHelper.internalServerError(res, `Error
interno.`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.internalServerError(res, `Error:
${error.message}`);
    }
});

router.get('/:id', async (req, res) => {
    try {
        let id = req.params.id;
        const returnEntity = await currentService.getByIdAsync(id);

```

```

        if (returnEntity != null){
            return ResponseHelper.ok(res, returnEntity);
        } else {
            return ResponseHelper.notFound(res, `No se encontro la
entidad (id:${id}).`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.internalServerError(res, `Error:
${error.message}`);
    }
});

router.post('', async (req, res) => {
    try {
        let entity = req.body;
        const newId = await currentService.createAsync(entity);
        if (newId > 0 ){
            return ResponseHelper.created(res, newId);
        } else {
            return ResponseHelper.badRequestJson(res, null);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.put('/:id', async (req, res) => {
    try {
        let id = parseInt(req.params.id);
        let entity = req.body;

        if (entity.id && parseInt(entity.id) !== id) {
            return ResponseHelper.badRequest(res, `El id de la URL
(${id}) no coincide con el id del body (${entity.id}).`);
        }

        entity.id = id;
    }
});

```

```

        const rowsAffected = await
currentService.updateAsync(entity);
        if (rowsAffected !== 0){
            return ResponseHelper.ok(res, rowsAffected);
        } else {
            return ResponseHelper.notFound(res, `No se encontro la
entidad (id:${id}).`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.delete('/:id', async (req, res) => {
    try {
        let id = req.params.id;
        const rowCount = await currentService.deleteByIdAsync(id);
        if (rowCount !== 0){
            return ResponseHelper.ok(res, null);
        } else {
            return ResponseHelper.notFound(res, `No se encontro la
entidad (id:${id}).`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.internalServerError(res, `Error:
${error.message}`);
    }
});

export default router;

```

cursos-controller.js

JavaScript

None

```

import { Router } from 'express';
import CursosService from '../services/cursos-service.js'

```



```

import ResponseHelper from '../helpers/response-helper.js'

const router = Router();
const currentService = new CursosService();

router.get('', async (req, res) => {
  try {
    console.log(`CursosController.get`);
    const returnArray = await currentService.getAllAsync();
    if (returnArray != null){
      return ResponseHelper.ok(res, returnArray);
    } else {
      return ResponseHelper.internalServerError(res, `Error interno.`);
    }
  } catch (error) {
    console.log(error);
    return ResponseHelper.internalServerError(res, `Error: ${error.message}`);
  }
});

router.get('/:id', async (req, res) => {
  try {
    let id = req.params.id;
    const returnEntity = await currentService.getByIdAsync(id);
    if (returnEntity != null){
      return ResponseHelper.ok(res, returnEntity);
    } else {
      return ResponseHelper.notFound(res, `No se encontro la entidad (id:${id}).`);
    }
  } catch (error) {
    console.log(error);
    return ResponseHelper.internalServerError(res, `Error: ${error.message}`);
  }
});

router.post('', async (req, res) => {
  try {

```

```

        let entity = req.body;
        const newId = await currentService.createAsync(entity);
        if (newId > 0 ){
            return ResponseHelper.created(res, newId);
        } else {
            return ResponseHelper.badRequestJson(res, null);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.put('/:id', async (req, res) => {
    try {
        let id = parseInt(req.params.id);
        let entity = req.body;

        if (entity.id && parseInt(entity.id) !== id) {
            return ResponseHelper.badRequest(res, `El id de la URL
(${id}) no coincide con el id del body (${entity.id}).`);
        }

        entity.id = id;
        const rowsAffected = await
currentService.updateAsync(entity);
        if (rowsAffected !== 0){
            return ResponseHelper.ok(res, rowsAffected);
        } else {
            return ResponseHelper.notFound(res, `No se encontro la
entidad (id:${id}).`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.delete('/:id', async (req, res) => {

```

```

    try {
      let id = req.params.id;
      const rowCount = await currentService.deleteByIdAsync(id);
      if (rowCount !== 0){
        return ResponseHelper.ok(res, null);
      } else {
        return ResponseHelper.notFound(res, `No se encontro la entidad (id:${id}).`);
      }
    } catch (error) {
      console.log(error);
      return ResponseHelper.internalServerError(res, `Error: ${error.message}`);
    }
  });
});

export default router;

```

materias-controller.js

## JavaScript

None

```

import { Router } from 'express';
import MateriasService from '../services/materias-service.js'
import ResponseHelper from '../helpers/response-helper.js'

const router = Router();
const currentService = new MateriasService();

router.get('/', async (req, res) => {
  try {
    console.log(`MateriasController.get`);
    const returnArray = await currentService.getAllAsync();
    if (returnArray !== null){
      return ResponseHelper.ok(res, returnArray);
    } else {
      return ResponseHelper.internalServerError(res, `Error interno.`);
    }
  } catch (error) {

```

```

        console.log(error);
        return ResponseHelper.internalServerError(res, `Error:
${error.message}`);
    }
});

router.get('/:id', async (req, res) => {
    try {
        let id = req.params.id;
        const returnEntity = await currentService.getByIdAsync(id);
        if (returnEntity != null){
            return ResponseHelper.ok(res, returnEntity);
        } else {
            return ResponseHelper.notFound(res, `No se encontro la
entidad (id:${id}).`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.internalServerError(res, `Error:
${error.message}`);
    }
});

router.post('', async (req, res) => {
    try {
        let entity = req.body;
        const newId = await currentService.createAsync(entity);
        if (newId > 0 ){
            return ResponseHelper.created(res, newId);
        } else {
            return ResponseHelper.badRequestJson(res, null);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.put('/:id', async (req, res) => {
    try {

```

```

    let id = parseInt(req.params.id);
    let entity = req.body;

    if (entity.id && parseInt(entity.id) !== id) {
        return ResponseHelper.badRequest(res, `El id de la URL
({{id}}) no coincide con el id del body ({{entity.id}}).`);
    }

    entity.id = id;
    const rowsAffected = await
currentService.updateAsync(entity);
    if (rowsAffected !== 0){
        return ResponseHelper.ok(res, rowsAffected);
    } else {
        return ResponseHelper.notFound(res, `No se encontro la
entidad (id:{{id}}).`);
    }
    } catch (error) {
        console.log(error);
        return ResponseHelper.badRequest(res, `Error:
${error.message}`);
    }
});

router.delete('/:id', async (req, res) => {
    try {
        let id = req.params.id;
        const rowCount = await currentService.deleteByIdAsync(id);
        if (rowCount !== 0){
            return ResponseHelper.ok(res, { mensaje : "Se elimino
con exito la materia" });
        } else {
            return ResponseHelper.notFound(res, `No se encontro la
entidad (id:{{id}}).`);
        }
    } catch (error) {
        console.log(error);
        return ResponseHelper.internalServerError(res, `Error:
${error.message}`);
    }
});

```

```
export default router;
```

## PROMPT 2:

### # ROL

Actúa como un Arquitecto de Software Senior y un revisor de código extremadamente riguroso,

### # CONTEXTO

En nuestra interacción anterior, me diste una solución donde delegabas la definición o el manejo de los HTTP Status Codes dentro de los Helpers, en lugar de mantenerlos en la capa de los Controllers. Esto es un error.

### # TAREA

Corrige tu ERROR . Reubica la responsabilidad de los Status Codes exclusivamente en los Controllers. El Helper de respuestas debe ser agnóstico a la lógica de negocio

### # RESTRICCIONES

Los Helpers NO pueden contener constantes de status codes (como 200, 404, 500) ni métodos acoplados a un estado específico (como `helper.sendNotFoundError`). Deben recibir el status code como un parámetro dinámico enviado por el Controller.

